

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/320557076>

Detecting Android Malwares By Mining Statically Registered Broadcast Receivers (Full paper)

Conference Paper · October 2017

DOI: 10.1109/CIC.2017.00021

CITATIONS

12

READS

102

4 authors, including:



Fadi Mohsen

University of Michigan-Flint

30 PUBLICATIONS 126 CITATIONS

SEE PROFILE



Halil Bisgin

University of Michigan-Flint

62 PUBLICATIONS 980 CITATIONS

SEE PROFILE

Detecting Android Malwares By Mining Statically Registered Broadcast Receivers

Fadi Mohsen*, Halil Bisgin*, Zachary Scott and Kyle Strait

Department of Computer Science, Eng. & Phy.

University of Michigan Flint

Flint, MI 48502, USA

Email: {mohsenf, bisgin, zascott, kstrait}@umflint.edu

Abstract—Android has a large share in the mobile apps market which makes it attractive for both malicious and good developers. Online apps markets, despite their vetting procedures, still admit malicious apps that could be downloaded mistakenly by mobile users. Detecting Android malwares has been studied by many researchers using different approaches and techniques. The vast majority of them though were focused on using the requested *permissions* that are declared in the *AndroidManifest.xml* files. A number of researchers have considered other components of the Android applications besides the permissions such as package info, activities, and process name. Some researchers pointed out Android *Broadcast receivers*’ component but it was not discussed thoroughly like other components. In this paper, we are conducting an empirical study to investigate the usage patterns of the Broadcast receivers component by malicious and benign Android applications. In addition to processing the *AndroidManifest.xml* files, the source code of malware samples, in particular the *onReceive()* are manually analyzed. We also propose a data mining malware detection mechanism based on the statically registered Broadcast receivers. Our research shows that Android Broadcast receivers are intensively used by malware compared to benign applications. Our Java code analysis shows that malware samples fully utilize Broadcast receivers compared to benign apps. Finally, our experiments showed that using the Broadcast receivers with permissions improves the malwares prediction accuracy.

I. INTRODUCTION

The number of mobile apps in the official and non-official online markets continues to grow at a fast pace. According to an International Data Corporation (IDC) forecast, mobile device users installed 156 billion mobile applications worldwide in 2015 and they estimates that this number will grow to more than 210 billion installs in 2020 and it will generate \$57 billion in direct revenue [21]. Because of the popularity of the mobile app markets, many companies are trying to invest and take advantage of it, for instance, Google announced Android Things that aims at creating a better Internet of Things (IoT) operating system [17]. Also, Microsoft announced Project Rome Android SDK, which enables users to control their Windows 10 laptops from their Android mobile devices. On the other hand, malwares that target mobile devices are also spreading and increasing in

numbers and abilities. According to a report by Kaspersky in Q1 2016 [1], their lab detected more than 2 million malicious installation packages which was 11 times greater than in Q4 2015, and 1.2 times more than in Q3 2015.

There have been many research efforts to detect Android malwares whether based on their behaviors [10], [28], [33], [34], signatures [22], [32], [26], [23], [25], [19], [31], [30], [20], [2] or both [3]. Behavior-based malware detection and analysis is done by monitoring system API calls, information flow tracking, and network monitoring, an approach that was proven to be effective, but it causes high overhead on mobile devices. On the contrary, the signature-based solution is light and relies on analyzing contextual information such as an app’s description, category, and users’ reviews, technical information such as requested permissions, activities, and package imports. Researchers have used different techniques to analyze this information such as calculating malignancy score [26], using weighted variation SVMs [29], applying probabilistic generative models (Naive Bayes) [24], using text mining algorithms [11], and conducting empirical studies [12].

However, researchers in their proposed approaches have paid less attention to the Android Broadcast receiver component [16]. A component that let apps register to listen to system events such as when an SMS is received, call is made, and connectivity is changed. We believe that this component is vital in determining the malicious behavior of an app. Thus, in this paper, we are focusing on investigating its usage patterns among malicious and benign application; furthermore, we are aiming at using data mining algorithms to study the feasibility of using Android Broadcast receivers alone or in conjunction with the permissions info to detect malicious apps.

There are a vast number of system broadcasts that some malicious applications could take advantage of. In so doing, we are first conducting an empirical study on the usage of Broadcast receivers by a dataset of 200 benign and 200 malicious Android applications by analyzing their *Manifest* files and Java source code files [14]. Then on a different dataset of 5723 malware samples and a matching 5723 benign applications selected from 48,262 apps, we first attempt to detect malicious Android applications based on their permissions request, then based on Broadcast receiver

* Authors contributed equally

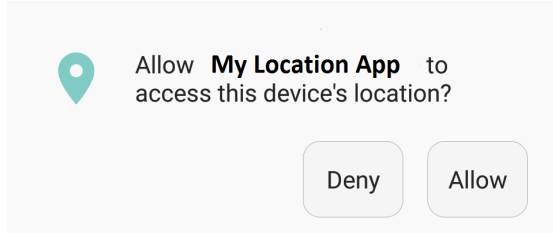


Figure 1. Android System displays this dialog prompting the user to grant or deny the location permission for the My Location App.

info, then based on both the permissions and Broadcast receiver.

The rest of the paper is organized as follows: Section II provides background information about Android apps components and their life cycles, Section III presents the results of our empirical study on a dataset of 400 malicious and benign apps, Section IV explains the methodology we used to detect malicious apps, Section VI discusses the related works, and we conclude the paper in Section VII.

II. PRELIMINARIES

The feature vector for each application is constructed by extracting the permissions and statically registered Broadcast receivers from the application's Manifest file. In this section we explain the Android permissions system, the *manifest* file and Android application's components and their life cycles.

A. Permissions

Android system uses sandboxing to separate apps from each other and from the system. If an app needs to make any access outside of its sandbox, it must specify that in its manifest file by listing the set of permissions it requires as shown in 1, the *ParseApp* is requesting two permissions, *CALL_PHONE* and *READ_SMS*. In the Android permission system, users upon installing apps into their mobile devices are prompted with the requested permissions that they need to approve in order for an app be installed and function properly. In fact, the Android system classifies permissions into normal and dangerous levels. For example, a permission such as *ACCESS_NETWORK_STATE* is considered normal whereas the *READ_CONTACTS* permission is considered dangerous since there is a risk to the user's privacy or security in letting apps have this permission. The Android system upon installing an app does not display the normal permissions, it only displays the dangerous ones for users to approve. Once the user approves the dangerous permission requests, the system installs the app and grants it all the permissions including the normal ones. Android has recently changed its permissions system, in particular what happens after the user approves the permissions [7]. The system takes user's upon-install approval final on less dangerous permissions; however, for the more dangerous permissions,

it may prompt the user at runtime to approve or reject the permission requests as it shown in Figure 1.

```

1 <manifest xmlns:android="http://schemas.
  android.com/apk/res/android"
2   package="com.android.app.ParseApp" >
3   <uses-permission android:name="android.
  permission.CALL_PHONE" />
4   <uses-permission android:name="android.
  permission.READ_SMS" />
5   ...
6
7
8   <receiver android:name=".
  ExternalBroadcastReceiver"
  android:exported="true">
9     <intent-filter>
10      <action android:name="android.intent.
  action.BOOT_COMPLETED" />
11      <action android:name="android.intent.
  action.ACTION_AIRPLANE_MODE_CHANGED" />
12    </intent-filter>
13  </receiver>
14
15  ...
16
17 </manifest>

```

Listing 1. A snapshot of an Android Manifest file

B. Android Application Components

An Android application is mainly composed of any combination of the following components: Activity, Service, Content Provider and Broadcast receiver. An activity is one of the fundamental building blocks of Android applications. It provides the graphical user interface to the end user to interact with it such as Buttons, Text Fields, Menus, etc. On the other hand, the service component normally runs in the background as long as the application is running and active. The content provider supplies the application with data such as the contact list. The Broadcast receiver component listens to system events and acts upon that. For instance, an application can register to listen to events such as "battery is low", "connection changed", and an "SMS received".

C. Components Lifecycles

When the user starts an Android application, interacts with, and then closes it, the Activity instances in the application transition through different states in their lifecycle. The Activity class provides a set of six callbacks to navigate the transitions between these states: *onCreate()*, *onStart()*, *onResume()*, *onPause()*, *onStop()*, and *onDestroy()* [4]. Android OS calls one of these callbacks whenever an activity changes its state. When the user decides to leave the activity, the system calls a method to kills the activity partially or completely. It is worth noting here that developers do not need to implement all of the six callbacks methods. Developers use the Service component [4] in their applications to perform long-running operations in the background. The Service does not provide a user interface, other

application components can start a service or bind to it, and it continues to run in the background even if the user decides to switch into another application. In Android 5.0, a new service type was introduced, called scheduled. The lifecycle of a Service depends on its type: Started, Bound, or Scheduled. The lifecycle of a Started service for example starts with a call to the *startService()* method, the service continues to run until it stops itself with *stopSelf()* or another component calls *stopService()*. For the bound service, if a component calls *bindService*, the service runs as long as there is a client bound to it. When all clients are unbound from the service, the system destroys it. There are scenarios where the Android system force-stops or kills a service, particularly if they are long-running and when memory is low.

The Broadcast component [4] allows applications to send or receive broadcast messages from the Android system or other applications when a particular event occurs. The system routes broadcasts to the subscribed applications. There are two ways through which an app can subscribe, statically in the Manifest file, or dynamically (context-registration) in the Java source code. The difference between them is that context-registered receivers can receive broadcasts as long as their registering context is active. For example, if a receiver is registered in an *Activity*, it would continue to receive broadcasts as long as the activity is active. On the other hand, statically registered receivers become a separate entry point into the app that the system can deliver the broadcast to even if it is not currently running. As shown in 1, the *ParseApp* statically registers a broadcast receiver for the actions: *BOOT_COMPLETED* and *ACTION_AIRPLANE_MODE_CHANGED*. In both cases, a class that subclasses *BroadcastReceiver* and implements *onReceive(Context, Intent)* must be created to handle the broadcasts when they are delivered to the app. The system creates a new object of the *BroadcastReceiver* class for each delivered broadcast, and after the code returns from the *onReceive(Context, Intent)* method, the object will be discarded by the system.

As it appears from the previous discussion, Broadcast receivers in general and statically registered ones in particular have an advantage over other application's components. They require less or no user interaction to be activated or triggered. Thus, they can properly be largely used by malicious apps to compromise users' privacy.

III. EMPIRICAL STUDY

In this section, we discuss the study that we have conducted on a dataset of 200 Android malware samples and 200 benign applications. The datasets were collected from Google Play store and other websites by the authors of [14] and by user submission. Two main analysis were performed, manifest analysis and Java source code analysis. The results are then summarized and compared among the two datasets.

Table I
THE NUMBER OF APPS THAT HAS A RECEIVER COMPONENT.

Category	Has Receiver	Has Receiver Sys. Action
Malware	74%	70%
Benign	65%	46%

Table II
THE TOP SYSTEM ACTION RECEIVED BY MALWARE SAMPLES.

#	Action	Frequency
1	BOOT_COMPLETED	131
2	USER_PRESENT	91
3	PHONE_STATE	88
4	SMS_RECEIVED	23
5	NEW_OUTGOING_CALL	9

A. Reverse Engineering

For the purpose of extracting the manifest files, the Apk-tool [13] is used. Apktool is a tool for reverse engineering Android third-party apps. It disassembles APS's resources to nearly original form including classes.dex, and XMLs. As per obtaining the Java source code, we first used the dex2jar tool [8] then the JD tool [15].

B. Analyzing The Manifest File

Android application package (APK) is the standard format used by the Android operating system for distributing mobile apps. We used reverse engineering tools on the dataset's APKs to retrieve the manifest files and the Java source codes. The manifest file contains information such as permissions, activities, services, broadcast receivers, etc. Upon installing Android apps onto mobile devices, Android *Package Manager* parses the manifest files and passes the necessarily information to the system. For example, it parses broadcast receivers information so that the system knows which apps to be notified when a certain event happens. From the manifest files we are mainly interested in retrieving the permissions and broadcast receivers of system actions. As far as the Java source code, to learn the difference between benign and malicious applications, the receivers' related parts are investigated and compared as we will explain later in this paper.

1) *Broadcast Receivers*: The manifest files of both datasets are processed to obtain statically registered broadcast receivers. In fact, we are only concerned with the receivers of the system actions. The set of system actions of Android OS releases are compiled in one file. The manifest files are parsed to find the occurrence of each one of them. The analysis revealed as it appears in Table I that 74% of malware samples have a statically registered receiver component in their manifests and 70% have a receiver component for a system action. On the other hand, the percentages are less for benign applications as 65% of them have a receiver component and only 46% of them have a receiver for a system action. Table III shows the

Table III
THE TOP SYSTEM ACTIONS RECEIVED BY BENIGN APPLICATIONS

#	Action	Frequency
1	BOOT_COMPLETED	86
2	PACKAGE_REPLACED	24
3	PACKAGE_ADDED	22
4	PACKAGE_REMOVED	18
5	TIME_SET	14

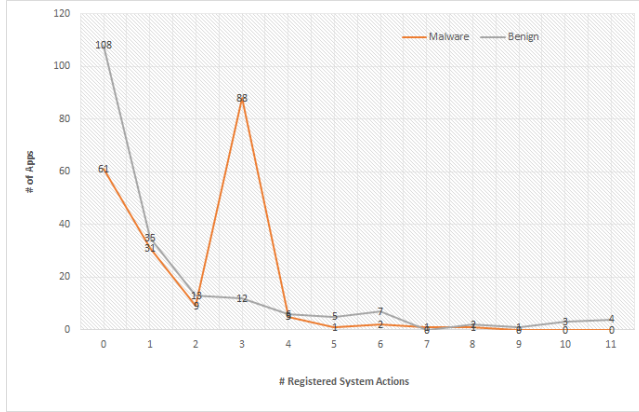


Figure 2. System Actions Distributions for Malware and Benign Applications

top 5 system actions most received by benign applications and Table II shows the same but for malware samples. Apparently, the BOOT_COMPLETED system action is the top most received by both samples. However, if you look at the rest of the actions there seem to be a noticeable difference between the two samples as the system actions that are received by malware samples are more sensitive and could pose privacy concerns to the mobile phone users. Another interesting finding is that the benign samples have collectively statically registered to receive a total of 57 distinct system actions compared to 28 for the malware samples. This finding suggests that malware are mainly concerned with registering to receive system actions that could compromise users privacy and help with executing their malicious payloads.

Figure 2 shows the number of system actions distributions by malware and benign applications. There are 61 malware applications that do not have a receiver for any system action in comparison to 108 benign applications. The biggest difference between the two datasets can be seen in the number of apps that has receivers for 3 system actions, 88 for the malicious and only 12 for the benign.

2) *Permissions*: There have been many works that relied on mining permissions requests to detect Android malicious apps. For example the work of [27] evaluated the capacity of permissions to detect malware using machine-learning techniques. They obtained high accuracy results but at the same time they had high false positive numbers. Thus, they suggested using other features besides the permissions. In

Table IV
THE TOP PERMISSIONS REQUESTED BY BENIGN APPLICATIONS

#	Permission	Frequency
1	INTERNET	159
2	ACCESS_NETWORK_STATE	135
3	WRITE_EXTERNAL_STORAGE	132
4	VIBRATE	87
5	READ_PHONE_STATE	85
6	WAKE_LOCK	74
7	RECEIVE_BOOT_COMPLETED	69
8	ACCESS_WIFI_STATE	61
9	CHECK_LICENSE	49
10	ACCESS_FINE_LOCATION	39

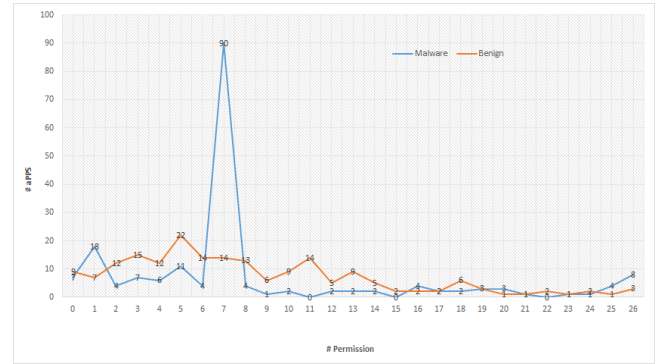


Figure 3. Number of Permissions Per App for Benign and Malicious Apps.

this work, we are extracting the permissions requests from the manifest files to be used as a stand-alone feature and as a secondary feature in our classification work. Figure 3 shows the number of permissions per app distribution for the malware and benign samples. As per the benign samples, the number of permissions per app ranges from 0 to 39 and the average is 9. On the other hand, the number of permissions ranges from 0 to 45 and the average is 9.5 for the malware samples. It is interesting to see that the vast majority of malware samples have exactly 9 permissions. Similar to the findings of Section III-B1 with regard to the system actions, the malware samples have collectively requested 98 distinctive permissions in comparison to 178 for the benign samples. Finally, Table IV and Table V displays the top 10 most requested permissions by benign and malware samples respectively. Surprisingly, there is 70% overlap between the two permissions sets. However, there are some interesting cases. By looking at the remaining 3 permissions of the malware list: For example, the SEND_SMS permission was requested by 67% of the malware samples in comparison to 5% of the benign samples, the INSTALL_PACKAGES permission, 56% of malwares and 0.5% of benign apps, and finally the DELETE_PACKAGES permission, 42% of malwares requested it in comparison to 0.0 % of benign apps. These results suggest that relying on Broadcast receivers for detecting Android malwares seems to guarantee better

Table V
THE TOP PERMISSIONS REQUESTED BY MALWARE APPLICATIONS

#	Permission	Frequency
1	INTERNET	189
2	READ_PHONE_STATE	161
3	ACCESS_NETWORK_STATE	132
4	WRITE_EXTERNAL_STORAGE	138
5	SEND_SMS	134
6	INSTALL_PACKAGES	111
7	DELETE_PACKAGES	83
8	WAKE_LOCK	82
9	ACCESS_WIFI_STATE	49
10	RECEIVE_BOOT_COMPLETED	46

results than relying on permissions alone. In this paper and based on the recommendations made by [27] both factors are going to be tested individually and combined.

C. Manual Analysis of Java Source Code

In this section we show the results of the manual analysis of the Java source codes for malicious and benign applications—that have statically registered Broadcast receivers. The aim of this analysis is to discover the difference between the two samples, in particular, we are looking to highlight the implementation patterns of the *onReceive()* by the malicious and benign applications. The system considers the process that hosts only a manifest-declared receiver to be a low-priority process, and upon returning from *onReceive()*, may kill it for performance reasons. Thus, developers are advised not to start long running background threads from a broadcast receiver as they might be terminated by the system. Though, there are other options to be used in such cases. For example, calling the *goAsync()* method [5] or schedule a *JobService* from the receiver using the *JobScheduler* [6]. In analyzing the source codes, the implementation of the *onReceive()* method is searched for an occurrence of any of the followings:

- Start a service : checking to see if the app upon receiving a broadcast starts a service. In particular, we are looking for any call to the following methods: *startService()* to start a service of type started, or *bindService()* for bound services , and *jobScheduler.schedule()* for scheduled services.
- Register a new broadcast receiver : sometimes, the Android OS limits delivering certain broadcasts to only context-registered receivers (e.g. the *CONNECTIVITY_ACTION*). Moreover, if many apps manifest-registers for the same broadcast, the system would have to launch all of them when the event that triggers that broadcast occur. Thus, it is possible to find apps context-register a new broadcast receiver inside the *onReceive()* of a manifest-registered receiver. This can be done by calling *registerReceiver()*.
- Call *goAsync()* to keep the broadcast active after returning from the *onReceive()* method as we explained

Table VI
THE RESULTS OF THE MANUAL ANALYSIS OF BENIGN AND MALICIOUS *onReceive* METHOD.

Action	Malware	Benign
Start Service	16%	42.4%
Register receiver	0%	5.43%
Call <i>goAsync</i>	0%	1.1%
Start Activity	65.5%	11%
Create Notification	11%	5.4%
None	7.5%	34.67%

in Section II-C.

- Call *startActivity()* to start a new activity.
- Call *notify()* to issue a notification.

The above analysis is done for both datasets, benign and malicious. The results could be utilized by static analysis tools such as [18]. Table VI shows the results of the manual analysis that was done on a total of 138 malicious and 92 benign apps. It was surprising to see that the vast majority of malicious apps, 65%, in their *onReceive()* method start an activity whereas 42.4% of benign applications found to start a service. Yet, 34.67% of the benign apps found to do none of the listed actions compared to only 7.5% of the malicious apps, which shows the true utilization of the Broadcast receivers of system actions by malicious apps.

IV. METHODOLOGY

In this section, we describe our data preprocessing steps, give an overview on Support Vector Machines (SVM) and discuss the parameter optimization for SVM through cross validation (CV) procedures. In Figure 4, we summarize these steps by which we establish our prediction pipelines for different scenarios that will be discussed in detail.

A. Data preprocessing

Our dataset consists of 48,262 benign and 5,723 malicious apps for which we parsed manifest files to extract action items and permissions. In order to avoid imbalanced class labels and to make use of benign apps at maximum efficacy, we created ten different balanced data sets by randomly selecting 5,723 benign apps to match the size of malicious apps as illustrated in Figure 4. Thus, we obtained ten different data sets, each having 11,446 apps. Since our dataset has action items and permissions, we considered three scenarios for the prediction stage: i) action items, ii) permissions, iii) action items and permissions combined.

We observed cases which shared exactly the same choices of action items and permissions. For this reason, we repeated the procedure above by only considering unique configurations which corresponds to the lower pipeline in Figure 4. In other words, we first picked only one of the repeating configurations for both malicious and benign apps. Therefore, the number of malicious and benign apps shrank to 960 and 8708, respectively. Due to the exclusive consideration of action items and permissions, we ended up with varying

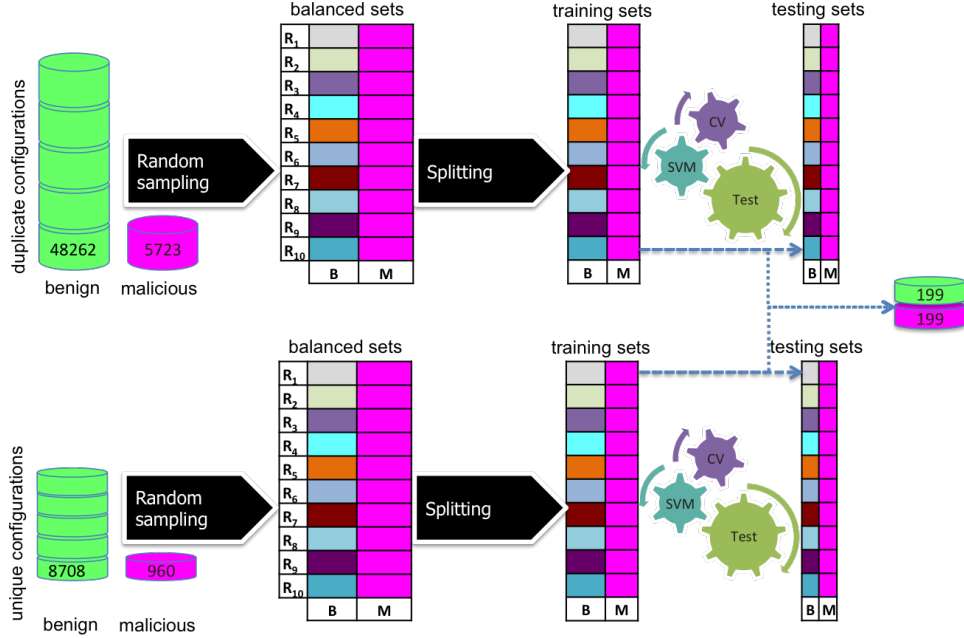


Figure 4. Flow of the study. Different colors in left columns represent randomly selected apps from the benign pool (green) and magenta in right columns represents malicious apps which are always the pool itself

numbers of sample sizes shown in Table VII for the three scenarios mentioned earlier. While it was straightforward for the former case where we always sampled 5,723 benign apps, we had to watch the varying numbers of malicious apps in the latter case. Namely, we further watched for the uniqueness as we exclusively worked with action items and permissions when sampling benign apps to match the counts of malicious apps. For instance, we sampled 83 benign apps to match the unique number of malicious apps when considering action items only, which resulted in balanced data sets with 166 apps. Finally, based on each criterion we randomly sampled from benign apps which resulted in ten balanced datasets for which we list the sizes in Table VII. For our experiments with SVM, we use those balanced datasets from which we form training and test sets as explained in Section IV-C.

Table VII
NUMBER OF APPS WHEN UNIQUE CONFIGURATIONS CONSIDERED

	benign	malicious	balanced dataset
actions	825	83	166
perms	7543	894	1788
total	8708	960	1920

B. Support Vector Machines

Our malware detection approach relies on support vector machines (SVMs) algorithm which was first introduced by Vapnik et al. [9] and believed to be a robust prediction method with further capacities. One of the major advantages

of SVMs is that linearly non-separable data can be classified by introducing *kernel* functions, which aid in drawing non-linear boundaries between classes in a higher dimensional space. With their linear and non-linear classification ability, SVMs have been extensively used in various fields such as text mining.

In a binary classification problem with class labels, $y_i \in \{-1, 1\}$, and the input data, $D = \{(x_i, y_i) | x_i \in \mathbb{R}^p\}_{i=1}^n$, hyperplanes can be defined as follows.

$$\begin{aligned} w^T x + b &\geq 1 \quad \text{if } y_i = 1 \\ w^T x + b &\leq -1 \quad \text{if } y_i = -1 \end{aligned} \quad (1)$$

SVM aims to maximize the distance (*margin*), $\rho = \frac{2}{\|w\|}$, between those hyperplanes that are drawn at the border of each class. Therefore, $\varphi(w) = \frac{1}{2}w^T w$ is minimized subject to $y_i(w^T x_i + b) \geq 1$.

In an extension of this quadratic optimization problem, a regularization parameter C is introduced and the resulting approach is coined as (C-SVM). We implement this approach in our study where we also seek for the optimum C value. The model can be further improved by incorporating the best γ value which is the tuning parameter in our kernel choice, radial basis function (RBF) expressed in Eq. 2.

$$K(x_i, x_j) = e^{-\gamma \cdot \|x_i - x_j\|^2} \quad (2)$$

C. Parameter optimization

After creating balanced datasets for the prediction stage, we split each dataset in Figure 4 with an approximately

4:1 ratio to designate training and testing sets, respectively. Since SVM classification requires parameter values for the kernel function, we first performed a 10-fold cross validation on the training sets to attain the best parameter combinations that would give us the highest prediction accuracies. Specifically, we sought for the optimum C values in $\{1E+01, 1E+02, 1E+03, 1E+04, 1E+05\}$ along with γ parameter in $\{1E-06, 1E-05, 1E-04, 1E-03, 1E-02, 1E-01\}$ which was due to our kernel choice, RBF.

Once we obtained the optimum parameter pairs for each case, we trained our model with those optimized pairs to predict malicious/benign behaviors on the test sets. Furthermore, as shown in Figure 4, we tested our models from two pipelines on an independent dataset for which we used the overlapping set of action items and permissions.

V. RESULTS

As we considered action items, permissions and their combined configurations, we performed a grid search to optimize γ and C values for each randomly generated dataset. Based on the average prediction accuracies, we found $1E-01$ and $1E+01$ as the optimum $\gamma - C$ pair for both duplicate and unique cases. Then we tested our optimized models on their corresponding testing sets that we had reserved earlier.

In Table VIII, we report the average accuracies and standard deviations for duplicate and unique cases for the randomly created 10 datasets along with changing features. It can be inferred from the table that action items have a potential to predict malicious/benign behavior with an average accuracy of 76%, but permissions seem to have higher capacity with 96% for the same task. Even though the prediction accuracy is higher for permissions, we observe another increase (97%) when action items are combined with permissions.

While the exclusive use of action items gives similar prediction performances for both cases, i.e., duplicate (78%) and unique (79%), we see a reduction from 96% to 86% in the latter case when we take the permissions into account. We observe a similar drop (97% vs. 86%) in the combined scenario where permissions are also involved. We believe that this performance difference is due to the fact that our classifier might be seeing exactly the same configuration during the training and the testing stages in the case of duplicates, which makes the prediction easier on the unseen data.

Table VIII
TESTING RESULTS

	duplicate conf.			unique conf.		
	Actions	Perms.	Comb.	Actions	Perms.	Comb.
avg	78%	96%	97%	79%	86%	86%
sd	0.006	0.005	0.004	0.065	0.018	0.013

In addition to the accuracies, we calculated the prediction power of those models at a more granular level. We

calculated the ratio of true positives (TP) to all positives (malicious) to obtain the sensitivity. Similarly, we took the ratio of true negatives (TN) to all negatives (benign) to report the specificity of models. In Table IX, we represent actual class labels with M and B whereas the predicted labels with $\hat{M}$ and $\hat{B}$ to determine true and false predictions.

Table IX
CONFUSION MATRIX

	$\hat{M}$	$\hat{B}$
M	TP	FN
B	FP	TN

In Figure 5, we summarize sensitivity and specificity values in box plots that illustrate the power of our ten different models along with other dimensions. As shown in the figure, action items can help in determining malicious behavior even though they favor the benign behavior more when predicting. In fact, the median values for sensitivity are 64% and 75% for duplicate and unique cases respectively. Although permissions give higher both sensitivity and specificity values, we notice the contribution of action items especially in sensitivity values when combined with permissions. Namely, action items help improving the distinguishing capacity for malicious behavior in both duplicate and unique cases.

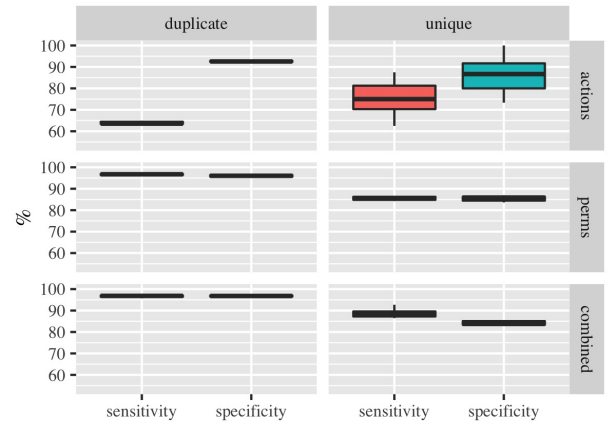


Figure 5. Sensitivity and specificity values for the testing sets

After testing our models in the reserved test sets, we moved to an independent dataset for which we extracted the same set of features, i.e., action items, permissions. The new dataset consisted of 398 apps where 199 of them were malicious. As we did in Table VIII, we used the best parameter sets and ran those ten different models developed on repeating (Duplicate Models) and unique configurations (Unique Models) on the new data. We present average accuracies along with the standard deviations in Table X where we have the minimum average accuracy of 71% that was obtained by using only action items.

Table X
TESTING RESULTS FOR NEW DATA

	Duplicate Models			Unique Models		
	Actions	Perms.	Comb.	Actions	Perms.	Comb.
avg	71%	76%	77%	72%	74%	76%
sd	0.003	0.010	0.007	0.01	0.01	0.01

We notice that our average accuracy rate is lower for the new test set when Table VIII and Table X are compared. For instance, we observe a drastic change from 97% to 77% in the case of permissions when duplicate configurations are allowed. However, use of action items gives close performances (78% vs. 71%) for the same scenario and have positive impact on prediction when utilized with permissions. The loss can be attributed to some discrepancies in the choice of action items and permissions in the new dataset, which is more recent than the initial one. Since we solely rely on the set of features from the former data, we only take into account the common features which result in some missing action items or permissions that only belong to the new set. Nevertheless, we observe much less difference in the accuracy rates between the models that were built on unique and repeating configurations within the later case, whereas in Table VIII we have nearly 10% loss on the average from duplicate to unique configurations. This concludes that despite the consistence models, existence of possibly new constituents in the recently developed apps appear to negatively affect our prediction ability.

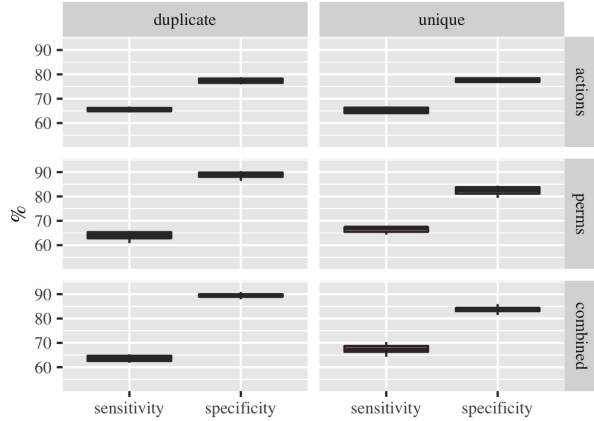


Figure 6. Sensitivity and specificity values for the new test set

We further illustrate the impact of missing features in Figure 6 where we summarize ten rounds of predictions in terms of sensitivity and specificity measures for both duplicate and unique models. Sensitivity and specificity values of models on the new test sets are aligned with the accuracy values. However, the lack of desired number of features, which were utilized in previous test sets, caused a decrease in both sensitivity and specificity levels. Moreover, the power of

classifying malicious apps accurately (sensitivity) was found to be lower than that of correctly predicting benign behavior (specificity) whereas models gave almost the same values for those measures in our earlier experiments. On the other hand, box plots show small variances for those measures, which is an indication of consistency across models.

VI. RELATED WORK

There have been many previous works on detecting malicious Android applications based on contextual information such as description and reviews and technical information such as permissions or any other information listed inside the manifest file. In this paper [32], they used the permission information extracted from the manifest file and the package imports extracted from the dex file to build their classifier. They used five classifiers: k-NN, Linear Discriminant Function, RBF Network, and Bayes. They tried detection based on permissions alone, package imports alone, and both. The authors of this work [26] used a malignancy score based on extracting six types of information from manifest files: Permission, Intent filter (action), Intent filter (category), Process name, Intent filter (priority), and number of redefined permissions. In this work [16], the researchers crafted some rules that describe malicious behaviors based on permissions combinations, they also mentioned broadcast receivers in one of their rules. In this paper [29] the following information were used: Permissions, Category of the app, Permissions requested by other apps in the same category, their work extended and compared with the work of [16] mentioned earlier. All of the aforementioned works have relied on technical information; on the other hand, the work of [22] in addition to permissions' technical information, contextual information such as description and reviews were only used. Our work is focused on using technical information extracted from the manifest file, mainly the permissions and receivers.

VII. CONCLUSION

Android malwares have become a serious problem to mobile users. Many researchers have worked on techniques to detect malwares and for that they utilized the information in the *AndroidManifest.xml* file. The requested permissions info was the most used among researchers. In this paper, we have focused on using another piece of information that is also declared in the *AndroidManifest.xml* file, called *Broadcast receivers* to detect Android malwares. In doing so, we have conducted an empirical study to investigate the usage patterns of the Broadcast receivers component by malicious and benign Android applications on a dataset of 200 benign and 200 malicious Android apps. We manually analyzed the source codes of the malware samples, in particular the *onReceive()* method. We also proposed a data mining malware detection mechanism based on the statically registered Broadcast receivers alone and compared it with

the mechanisms of using permissions alone and using both permissions and Broadcast receivers. Our research shows that Android Broadcast receivers are intensively used by malware compared to benign applications. The results of processing the *AndroidManifest* files showed that 74% of malware samples have one or more statically registered receiver and 70% have one or more receiver component for a system action. The percentages are lower for benign apps with 65% and 46%. The Java code analysis showed that 65% of the malicious apps in their *onReceive* method started an activity whereas 42.4% of benign apps started a service. The results showed that malicious apps were utilizing the Broadcast receiver way more than benign apps; 65.33% of benign apps have an implementation for the *onReceive* method compared to 92.5% for the malicious apps. Finally, our experiments showed that using the Broadcast receivers with permissions increased the malwares prediction accuracy to 97%.

REFERENCES

- [1] G. A., U. R., G. M., M. D., , and I. A. In threat evolution in q1 2016, 2016.
- [2] A. Amamra, C. Talhi, , and J.-M. Robert. Smartphone malware detection: From a survey towards taxonomy. In *2012 7th International Conference on Malicious and Unwanted Software (MALWARE)*, MALWARE'12. IEEE, 2012.
- [3] J. T. Andoor. *A Filtering Based Android Malware Detection System for Google PlayStore*, pages 559–566. Springer International Publishing, Cham, 2015.
- [4] Android. App Components. "<https://developer.android.com/guide/components/index.html>", February 2017.
- [5] Android. goAsync. "[https://developer.android.com/reference/android/content/BroadcastReceiver.html#goAsync\(\)](https://developer.android.com/reference/android/content/BroadcastReceiver.html#goAsync())", February 2017.
- [6] Android. JobScheduler. "<https://developer.android.com/reference/android/app/job/JobScheduler.html>", February 2017.
- [7] Android. Requesting Permissions at Run Time. "<https://developer.android.com/training/permissions/requesting.html>", February 2017.
- [8] Apache. dex2jar, 2017.
- [9] B. E. Boser, I. M. Guyon, and V. N. Vapnik. A training algorithm for optimal margin classifiers. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, COLT '92, pages 144–152, New York, NY, USA, 1992. ACM.
- [10] I. Burguera, U. Zurutuza, and S. Nadjm-Tehrani. Crowddroid: Behavior-based malware detection system for android. In *Proceedings of the 1st ACM Workshop on Security and Privacy in Smartphones and Mobile Devices*, SPSM '11, pages 15–26, New York, NY, USA, 2011. ACM.
- [11] Y. Chen, H. Xu, Y. Zhou, and S. Zhu. Is this app safe for children?: A comparison study of maturity ratings on android and ios applications. In *Proceedings of the 22Nd International Conference on World Wide Web*, WWW '13, pages 201–212, New York, NY, USA, 2013. ACM.
- [12] P. H. Chia, Y. Yamamoto, and N. Asokan. Is this app safe?: A large scale study on application permissions and risk signals. In *Proceedings of the 21st International Conference on World Wide Web*, WWW '12, pages 311–320, New York, NY, USA, 2012. ACM.
- [13] Connor Tumbleson and Ryszard Wisniewski. Apktool, 2017.
- [14] M. Damshenas, A. Dehghantanha, K.-K. R. Choo, and R. Mahmud. M0droid: An android behavioral-based malware detection model. *Journal of Information Privacy and Security*, 11(3):141–157, 2015.
- [15] Emmanuel Dupuy. JD Project, 2017.
- [16] W. Enck, M. Ongtang, and P. McDaniel. On lightweight mobile phone application certification. In *Proceedings of the 16th ACM Conference on Computer and Communications Security*, CCS '09, pages 235–245, New York, NY, USA, 2009. ACM.
- [17] Google. Iot-based android things developer preview 2 released, 2017.
- [18] M. I. Gordon, D. Kim, J. H. Perkins, L. Gilham, N. Nguyen, and M. C. Rinard. Information flow analysis of android applications in droidsafe. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015*, 2015.
- [19] S. K. Hannah Quay-de la Vallee, Paige Selby. On a (per)mission: Building privacy into the app marketplace. In *6th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices*, SPSP '16. ACM, 2016.
- [20] S. K. Hannah Quay-de la Vallee, Paige Selby. A survey on various malware detection techniques on mobile platform. In *International Journal of Computer Applications*, 2016.
- [21] I. D. C. (IDC). Mobile app revenue outlook remains healthy despite slowing download volumes and smartphone growth, according to idc, 2016.
- [22] Y. Jing, G.-J. Ahn, Z. Zhao, and H. Hu. Riskmon: Continuous and automated risk assessment of mobile applications. In *Proceedings of the 4th ACM Conference on Data and Application Security and Privacy*, CODASPY '14, pages 99–110, New York, NY, USA, 2014. ACM.
- [23] X. Li, J. Liu, Y. Huo, R. Zhang, and Y. Yao. An android malware detection method based on androidmanifest file. In *2016 4th International Conference on Cloud Computing and Intelligence Systems*, CCIS '16. IEEE, 2016.
- [24] H. Peng, C. Gates, B. Sarma, N. Li, Y. Qi, R. Potharaju, C. Nita-Rotaru, and I. Molloy. Using probabilistic generative models for ranking risks of android apps. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, CCS '12, pages 241–252, New York, NY, USA, 2012. ACM.

- [25] X. Ping, W. Xiaofeng, N. Wenjia, Z. Tianqing, and L. Gang. Android malware detection with contrasting permission patterns. In *China Cummunications*. IEEE, 2014.
- [26] D. C. Ryo Sato and S. Goto. Detecting android malware by analyzing manifest files. In *Proceedings of the Asia-Pacific Advanced Network 2013*, APAN '13, pages 23–31, 2013.
- [27] B. Sanz, I. Santos, C. Laorden, X. Ugarte-Pedrero, P. G. Bringas, and G. Álvarez. *PUMA: Permission Usage to Detect Malware in Android*, pages 289–298. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.
- [28] A. Saracino, D. Sgandurra, G. Dini, and F. Martinelli. Madam: Effective and efficient behavior-based android malware detection and prevention. *IEEE Transactions on Dependable and Secure Computing*, PP(99):1–1, 2016.
- [29] B. P. Sarma, N. Li, C. Gates, R. Potharaju, C. Nita-Rotaru, and I. Molloy. Android permissions: A perspective combining risks and benefits. In *Proceedings of the 17th ACM Symposium on Access Control Models and Technologies*, SACMAT '12, pages 13–22, New York, NY, USA, 2012. ACM.
- [30] V. F. Taylor and I. Martinovic. Securank: Starving permission-hungry apps using contextual permission analysis. In *Proceedings of the 6th Workshop on Security and Privacy in Smartphones and Mobile Devices*, SPSM '16, pages 43–52, New York, NY, USA, 2016. ACM.
- [31] Y. Wang, J. Zheng, C. Sun, and S. Mukkamala. Quantitative security risk assessment of android permissions and applications. In *Proceedings of the 27th International Conference on Data and Applications Security and Privacy XXVII*, DB-Sec'13, pages 226–241, Berlin, Heidelberg, 2013. Springer-Verlag.
- [32] XIANGYU-JU. Android malware detection through permission and package. In *Proceedings of the 2014 International Conference on Wavelet Analysis and Pattern Recognition*, ICWAPR '14. IEEE, 2014.
- [33] L. Xie and J. pierre Seifert. S.: pbmds: a behavior-based malware detection system for cellphone devices. In *In: 3rd ACM Conference on Wireless Network Security (WiSec 2010)*, pages 37–48. ACM, 2010.
- [34] S. Zou, J. Zhang, and X. Lin. An effective behavior-based android malware detection system. *Sec. and Commun. Netw.*, 8(12):2079–2089, Aug. 2015.